30-MINUTE STUDENT TASK

Build Your Own Rule-Based Python Chatbot

Student Name	P.Yashwanth Kumar
Registration Number	99230040738
Task	Rule-Based Conversational Chatbot in Python
Approach	Input → Process → Respond → Repeat → Exit

Department: CSE

1. Understanding the Task

The objective is to create a simple conversational program that does not depend on an AI model or external service. Instead, it uses clearly defined rules to interpret the user's message and select an appropriate response. The conversation continues until the user enters an exit command.

Implementation decisions

- The user's name is collected before the main conversation begins.
- Every message is normalized using **lower().strip()**, making the program less sensitive to capitalization and extra spaces.
- The **in** operator allows natural variations such as “hello bot” to be recognized as greetings.
- **if / elif / else** provides the decision-making structure.
- **while True** maintains the conversation, while **break** provides a controlled exit.
- An **else** response handles messages that are outside the chatbot's known rules.

The solution follows the exact target flow specified in the task: **INPUT** → **PROCESS** → **RESPOND** → **REPEAT** → **EXIT**. ■filecite■turn0file0■L37-L40■

2. Complete Python Implementation

The following program is written specifically for this task. It keeps the logic simple enough to understand and demonstrate, while adding enough conversational cases to show that the chatbot is more than a two-response example.

```
# Rule-Based Python Chatbot
# Student: P.Yashwanth Kumar
# RegNo: 99230040738

print("=== Welcome to Yashwanth's  Rule-Based Chatbot ===")
name = input("Before we begin, what is your name? ").strip()

print(f"Hello, {name}! I am a simple rule-based chatbot.")
print("You can ask me about Python, my capabilities, or how I am.")
print("Type 'bye' or 'exit' whenever you want to end the conversation.\n")

while True:
    user_input = input(f"{name}: ").lower().strip()

    if "bye" in user_input or "exit" in user_input:
        print("Bot: Goodbye! It was nice talking with you.")
        break

    elif "hello" in user_input or "hi" in user_input or "hey" in user_input:
        print(f"Bot: Hello, {name}! Nice to meet you. How can I help you today?")

    elif "how are you" in user_input:
        print("Bot: I'm doing great! Thanks for asking. I'm ready to help.")

    elif "what is python" in user_input or "python" in user_input:
        print("Bot: Python is a high-level programming language known for its clear syntax and wide range

    elif "what can you do" in user_input or "help" in user_input:
        print("Bot: I can answer basic questions about Python, learning, and general topics. Try asking me

    elif "time" in user_input:
        print("Bot: I cannot access the live clock, but you can check the current time on your device.")

    elif "thank" in user_input:
        print("Bot: You're welcome! I'm glad I could help.")

    else:
        print("Bot: I'm not sure I understand that yet. Try asking about Python, my capabilities, or how I
```

Why this implementation is robust

The exit condition is checked before the other conversational rules, so the user can always leave the program. Greeting detection uses the **in** operator rather than exact equality, which handles phrases containing a greeting. The remaining branches cover multiple topics, and the final **else** gives a useful recovery response instead of failing silently.

3. Logic and Concept Explanation

The chatbot can be understood as five small stages:

Stage	What happens	Python concept
1. INPUT	The user provides their name and then enters messages.	**input()**
2. PROCESS	The message is converted to lowercase and stripped of unnecessary spaces.	**.lower().strip()**
3. DECISION	The program checks the message against known rules in order.	**if / elif / else**
4. RESPOND	A matching response is printed to the screen.	**print()**
5. REPEAT / EXIT	The loop continues until “bye” or “exit” is detected.	**while True + break**

Conversation rules

1. **Greetings:** recognizes “hello”, “hi”, and “hey”, including phrases such as “hello bot”.
2. **Well-being:** responds to “how are you”.
3. **Python topic:** explains what Python is when the message contains “python”.
4. **Capabilities/help:** explains what the chatbot can answer.
5. **Time:** gives a transparent response instead of pretending to know the live time.
6. **Thanks:** acknowledges appreciation naturally.
7. **Exit:** “bye” and “exit” end the conversation.
8. **Unknown input:** the final else branch tells the user what kinds of questions they can try.

This structure directly satisfies the assignment's requirement for at least two greetings, at least three topics, conditional decision-making, an exit mechanism, name usage, and a useful unknown-input response. ■filecite■turn0file0■L13-L23■

4. Testing and Expected Results

The assignment explicitly asks for tests involving different capitalization, greetings, a question, an unknown message, and an exit command. ■filecite■turn0file0■L27-L36■ The cases below cover those checks and add a few practical variations.

Input	Purpose	Expected chatbot behavior
hello	Basic greeting	Hello, Yashwanth! Nice to meet you. How can I help you today?
HELLO	Case-insensitivity	Hello, Yashwanth! Nice to meet you. How can I help you today?
hi	Second greeting	Hello, Yashwanth! Nice to meet you. How can I help you today?
What is Python?	Question/topic	Python is a high-level programming language known for its clear syntax and wide range of applications.
hello bot	Flexible greeting using in	Hello, Yashwanth! Nice to meet you. How can I help you today?
How are you?	Conversation topic	I'm doing great! Thanks for asking. I'm ready to help.
random question	Unknown-message handling	I'm not sure I understand that yet. Try asking about Python, my capabilities, or how I am.
bye	Exit command	Goodbye! Have a productive day.
EXIT	Case-insensitive exit	Goodbye! Have a productive day.

Testing conclusion

The test set demonstrates that the chatbot continues to respond correctly when capitalization changes, recognizes a greeting embedded in a longer phrase, answers multiple topics, handles an unknown message gracefully, and terminates when an exit command is entered.

The core normalization step is important because both “HELLO” and “hello” become the same internal value after **lower().strip()**. This is a small design choice, but it makes the interaction noticeably more user-friendly.

5. Final Submission Summary

This chatbot demonstrates the fundamentals of a rule-based conversational system without hiding the logic behind a library. Each response is traceable to a condition, making the program easy to read, debug, extend, and explain during a demonstration.

Requirements checklist

Requirement	Implemented
Ask user's name before main conversation	Yes
Use while True	Yes
Use input()	Yes
Clean input with lower().strip()	Yes
Recognize at least two greetings	Yes — hello, hi, hey
Answer at least three questions/topics	Yes — Python, capabilities, well-being, time, thanks
Use if / elif / else	Yes
Use break for bye/exit	Yes
Use user's name in a response	Yes
Useful else response for unknown input	Yes
Use in for flexible variations	Yes

Demonstration points

- Explain that the chatbot is **rule-based**, so its behavior is determined by conditions rather than machine learning.
- Demonstrate that **HELLO**, **hello**, and **hello bot** can reach the greeting rule.
- Ask a Python-related question to show topic recognition.
- Enter an unrelated sentence to demonstrate the fallback **else** branch.
- Finish with **bye** or **exit** to demonstrate controlled termination.

Final note: The implementation is intentionally transparent and extensible. New topics can be added by inserting another elif rule with a clear condition and response, while preserving the same conversational loop.

Submitted by

P.Yashwanth Kumar
RegNo: 99230040738